

04

Building Agents

Overseeing the end-to-end ecosystem

Eleven ways agents coordinate — and what has to exist before a fleet runs beyond the team that built it.

The programme

Four sessions, one thread: from one working agent to a governed fleet.

DAY 1

Fundamentals

What an agent is, how it runs, where it belongs.

DAY 2

Processes

One judge became a pipeline that runs unattended.

DAY 3

Testing

Agents wrote the tests.
Grounding decided quality.

DAY 4

Ecosystem

Agents coordinate — and get owners, identities, records.

Every agent this week ran alone, on this machine, for this room.

Today is what changes when they stop being demos: they coordinate with each other, and they run where the people who built them are not watching.

Today: how agents work together — and what oversight has to exist before they do.

The week so far

Three machines, one shape.

DAY 2 • THE JUDGE

```
fetch -> rules_gate -> judge  
-> gate_ask -> post_review
```

*a person wired in where
consequence lives*

DAY 3 • THE TESTERS

```
design_tests -> run_suite  
-> triage_failures -> audit
```

*grounded in the contract;
checked by machinery*

DAY 3 • THE KNOWER

```
question -> search_knowledge  
-> grade -> answer + sources
```

*retrieve first,
reason second*

Inside every one of them: the same loop, and the same graph.

StateGraph: nodes that work, edges that route, state that flows. Today never leaves it.

Today puts many of these loops together — with the same `add_node` and `add_edge` you already know.

Today — patterns, then governance

Eleven ways to coordinate, three families. Then the fleet gets governed.

FIRST · ELEVEN PATTERNS, LIVE

the primitive	an agent calling another agent
fixed flow	sequential · concurrent · generator-verifier
a deciding agent	handoff · group chat · orchestrator · magentic
decoupled	agent teams · message bus · shared state

one use case, one graph, one command — per pattern

THEN · THE FLEET, GOVERNED

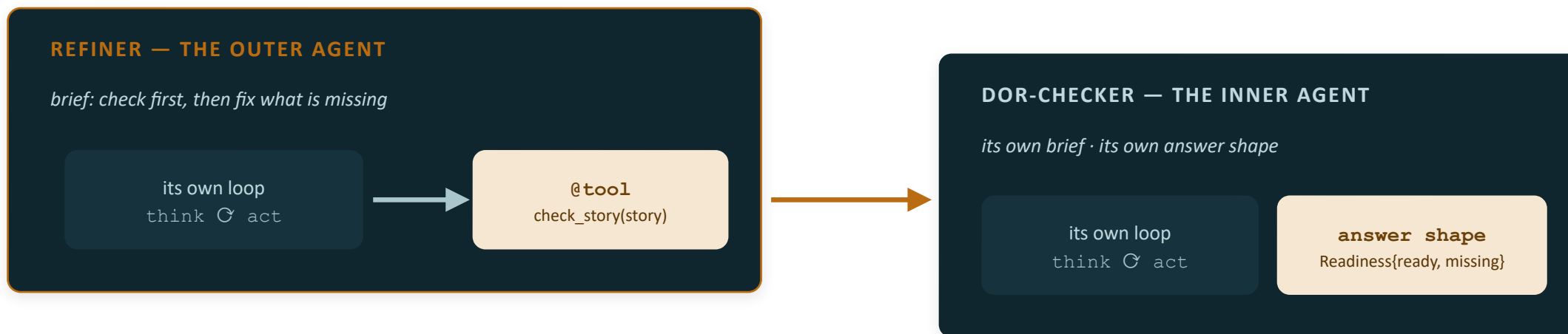
inventory	every agent: an owner, an identity of its own
run records	one line per run — and the agent that was quietly wrong
fleet view	every agent this week, one screen: cost, outlier, one proved change

the demos you run today land on that screen

Each demo is ONE self-contained file. Small fast model. Every run leaves a record.

The primitive — an agent calling another agent

Everything today stands on one trick: a whole agent, wrapped as a tool.



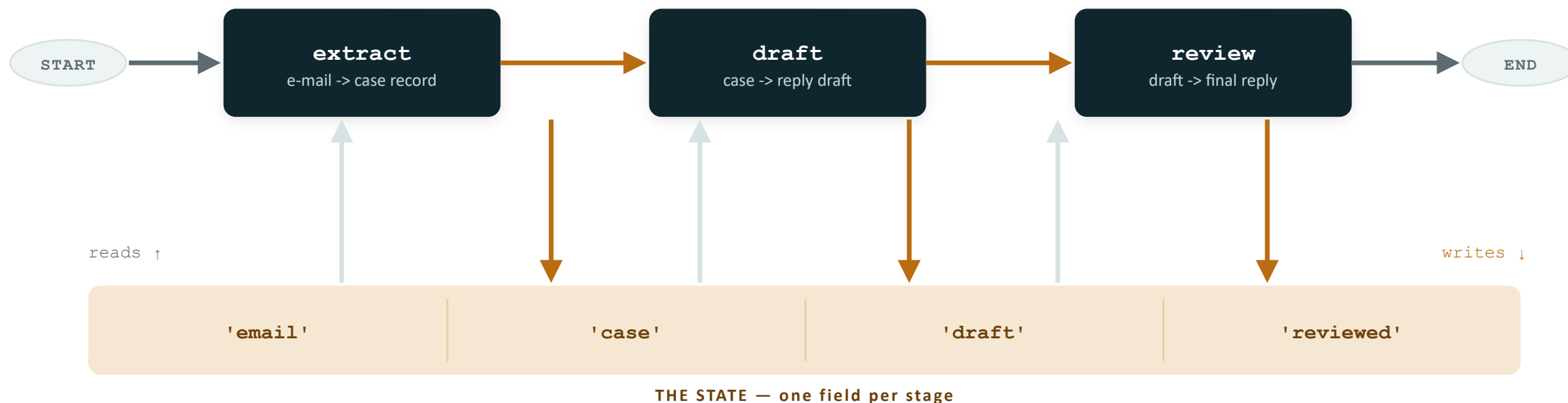
create_agent returns a compiled LangGraph — so this is a graph running inside another graph's loop. Neither knows the other's prompt, model or shape.

Every pattern today is this one trick, arranged differently.

DEMO `python p0_agent_calls_agent.py · story_refiner + Definition-of-Ready checker`

Sequential — a pipeline of agents

Fixed flow. The state is the conveyor belt; every node reads one field and writes the next.



What one node writes is exactly what the next node reads — output becoming input, three times. The order was drawn as edges, before any model ran.

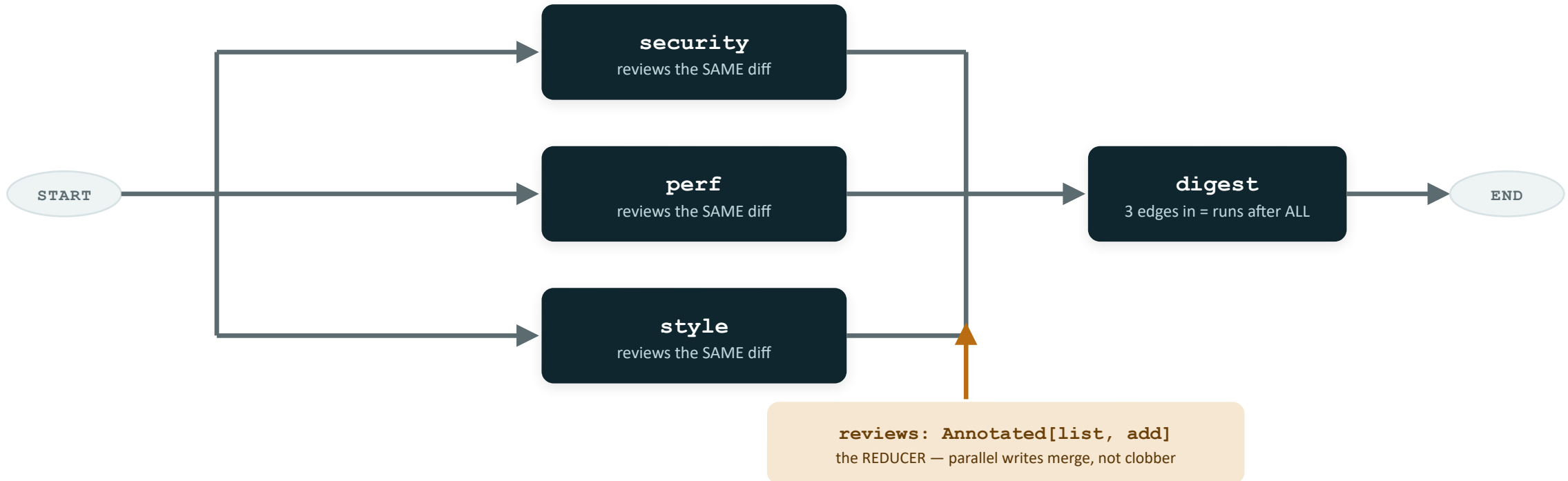
Zero coordination decisions at runtime — the shape was decided when the edges were drawn.

DEMO `python pl_sequential.py · support-mail intake: e-mail -> case -> draft -> reviewed reply`

Concurrent — parallel branches, one superstep

Fixed flow. Independent work fans out; a reducer merges the writes; the join is free.

ONE SUPERSTEP — RUN IN PARALLEL



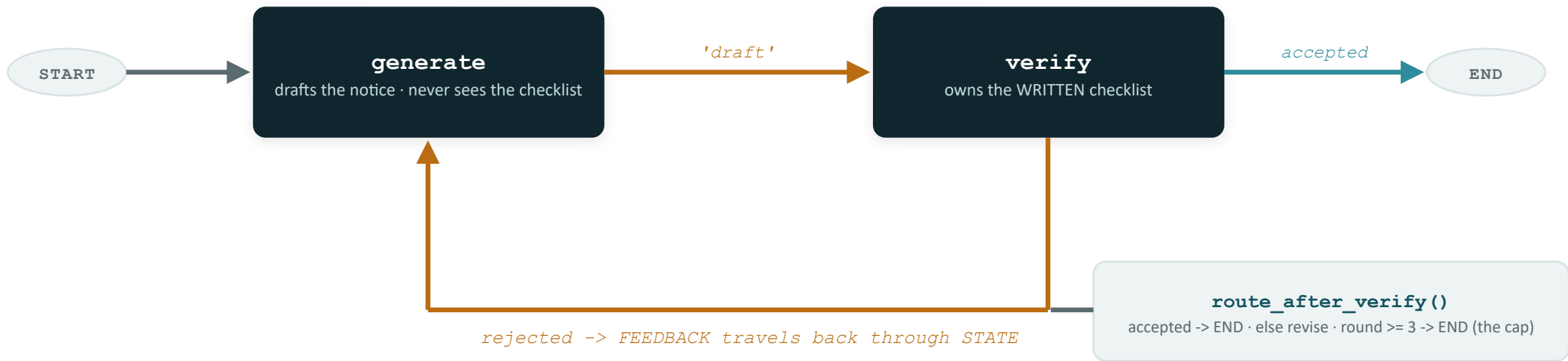
No thread pool, no locks — the graph owns the concurrency. Three lenses, none needs the others' output.

Agent time overlaps: ~7s of work inside ~3s of wall clock — drawn at build time.

DEMO `python p2_concurrent.py` · one PR diff, three review lenses at once

Generator-verifier — a quality loop, drawn

Fixed flow. The cycle is a conditional edge in the graph, not a while-loop in Python.



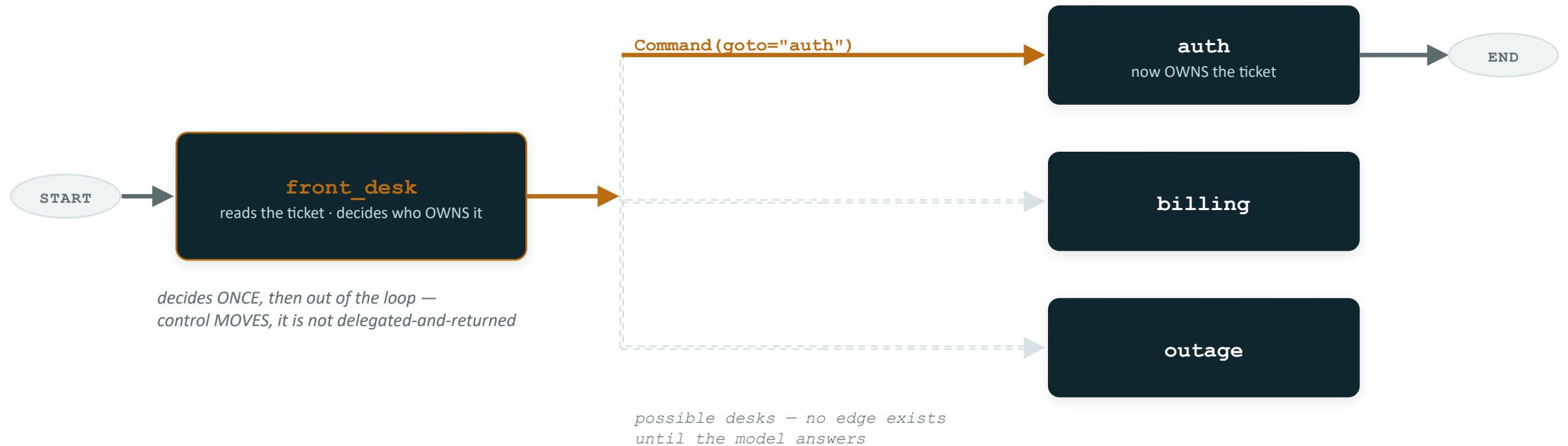
The loop is DRAWN: one conditional edge either closes the cycle or exits. The round cap is a written condition inside the routing function.

The verifier is only as strict as its written checklist — vague criteria, vague quality.

DEMO `python p3_generator_verifier.py · customer incident notice vs the comms checklist`

Handoff — the deciding agent routes the graph

A deciding agent. `Command(goto=...)`: the model's answer IS the edge.



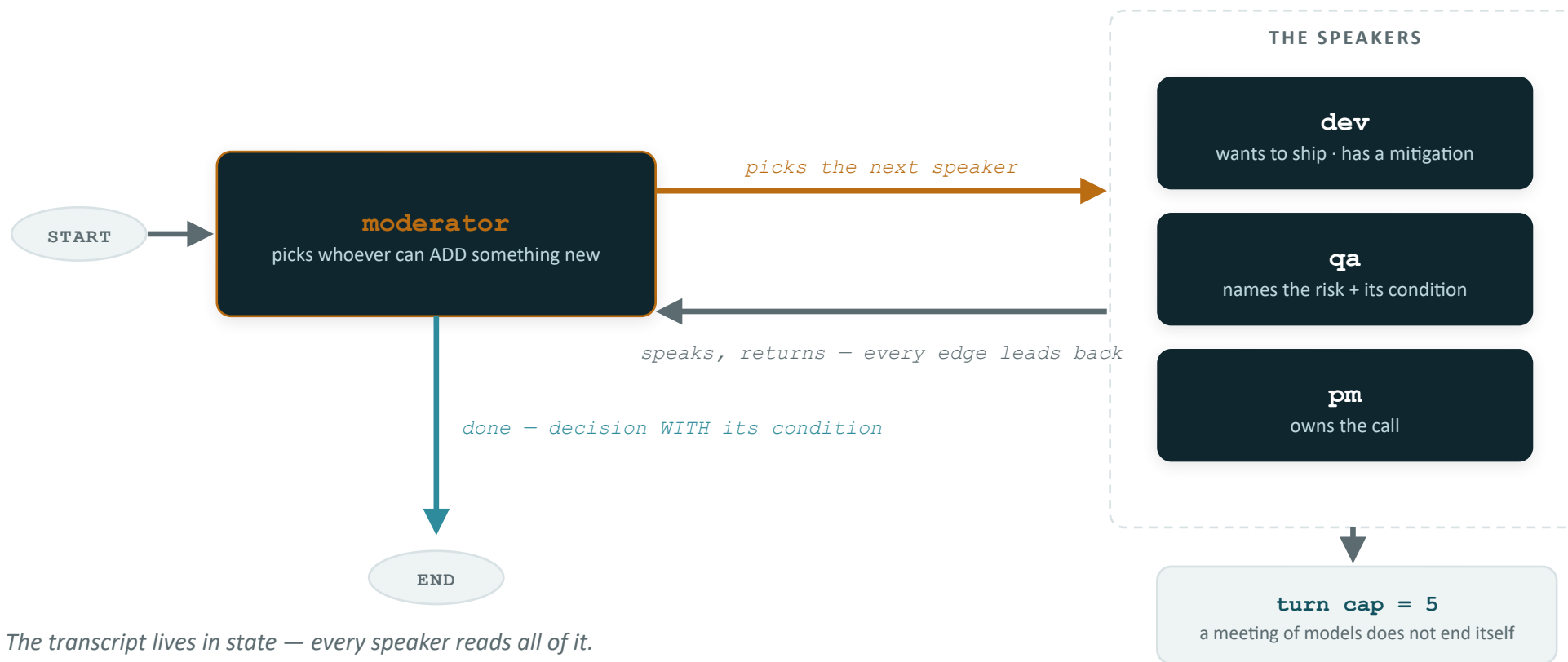
Compare day 2: there, a routing FUNCTION read the state and picked the edge. Here the deciding AGENT picks it — its structured answer becomes the route.

The transfer and its reason land in the state — auditable long after the ticket closes.

DEMO `python p4_handoff.py` · a support ticket routed to the desk that owns it

Group chat — a moderated cycle

A deciding agent. Every voice returns to the moderator; a written cap ends the meeting.



The transcript lives in state — every speaker reads all of it.

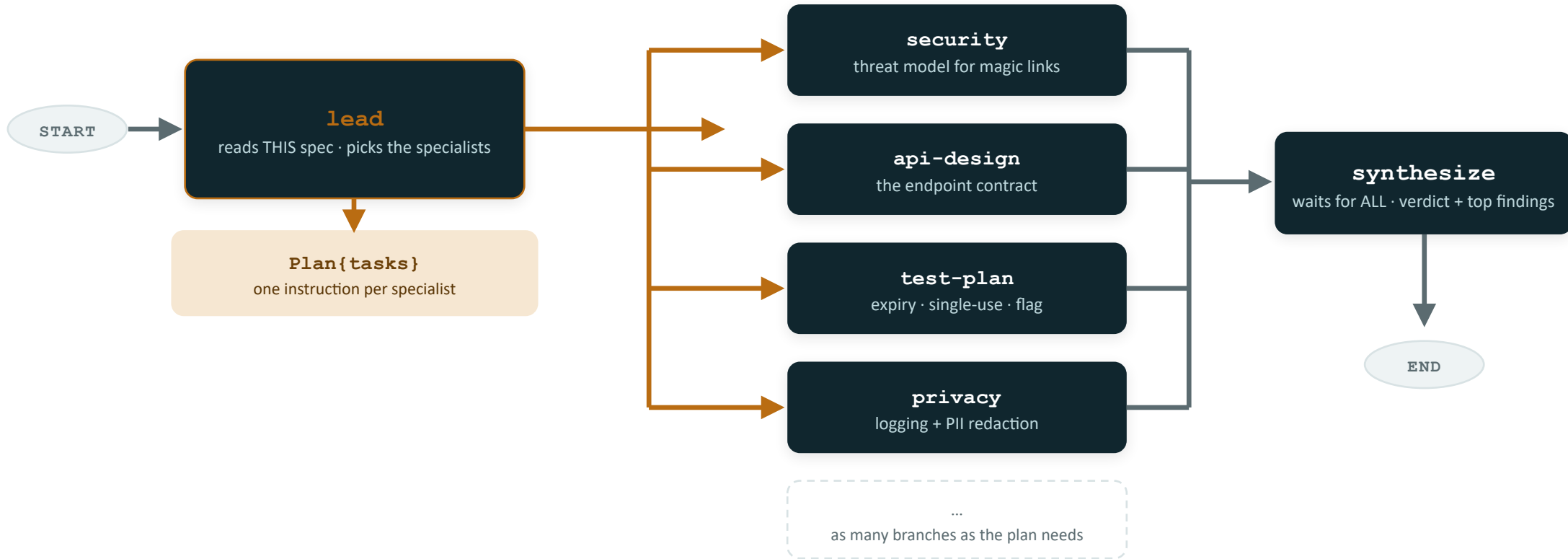
Costliest tokens per decision on this slide — reserve it for calls that deserve an argument.

DEMO `python p5_groupchat.py · ship Friday? dev, QA and PM hold real, conflicting stakes`

Orchestrator-subagent — branches created at run time

A deciding agent. Send: the lead's plan creates the branches. Anthropic's default pattern.

Send × N — CREATED AT RUN TIME, IN PARALLEL



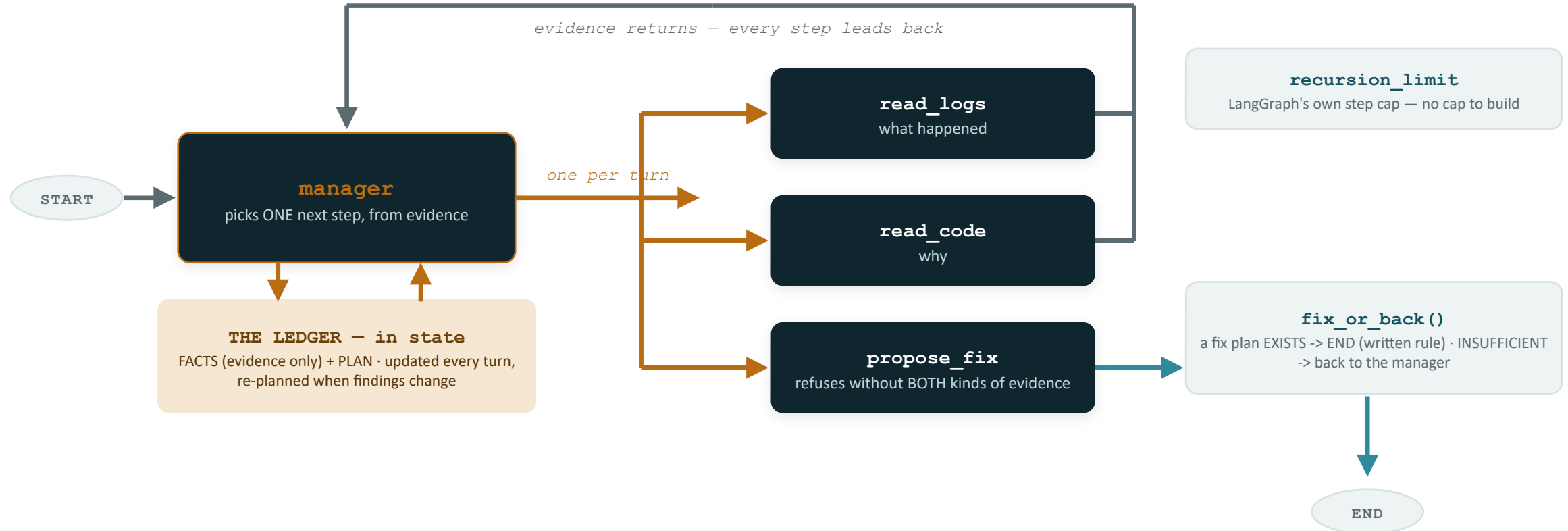
P2's three branches were drawn at BUILD time and never change. These do not exist until the lead answers — different spec, different fan-out, same graph.

Plan, delegate in parallel, synthesize — the default. Weakness: the lead is a bottleneck.

DEMO `python p6_orchestrator.py · a feature spec -> the checks THIS spec needs`

Magentic — a manager with a ledger

A deciding agent. Fixed edges, model-chosen path — and completion is a written rule.



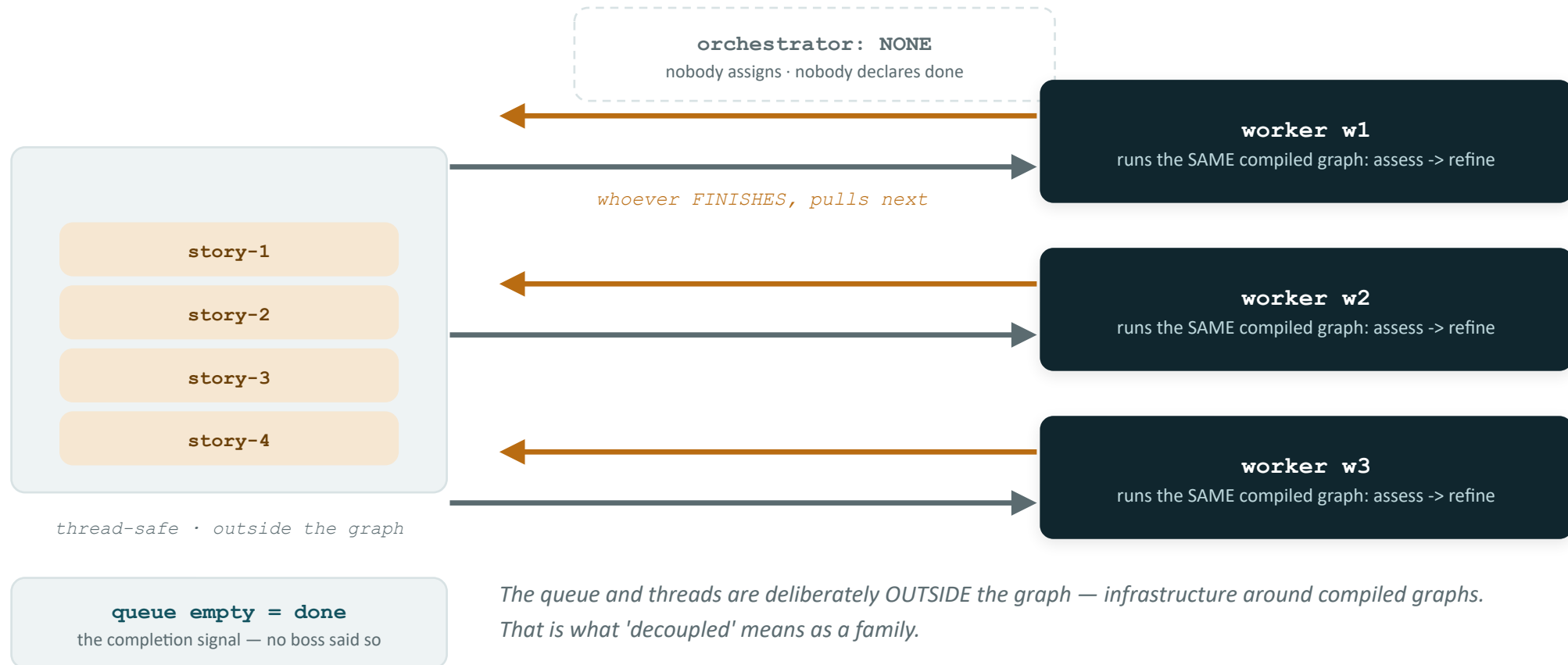
The edges never change; the PATH through them is the model's, chosen step by step from the ledger. Judgement to the model, rules to the graph.

For open-ended work where the step list cannot be known in advance — the most autonomous pattern today.

DEMO `python p7_magentic.py` · a bug investigation: double charge, cause unknown

Agent teams — a queue, workers, no boss

Decoupled. Persistent workers PULL from a shared queue; pace decides the split.



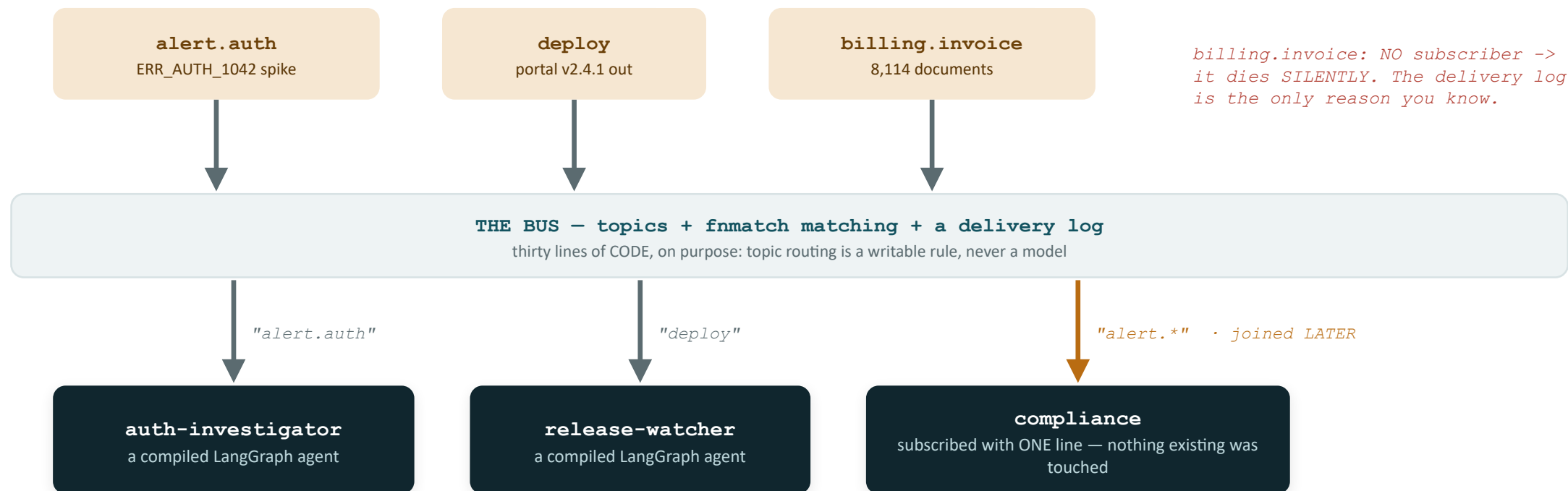
Many similar items, uneven pace: the split is decided by pace, not by judgement.

DEMO `python p8_agent_teams.py` · a backlog sweep: four stories, three workers

Message bus — publish, subscribe, never meet

Decoupled. Publishers don't know who listens; subscribers don't know who published.

EVENTS — published, sender unknown to listeners



Extend the system by subscribing — publishers, wires and existing agents never move. The bus connects graphs; it is deliberately not one.

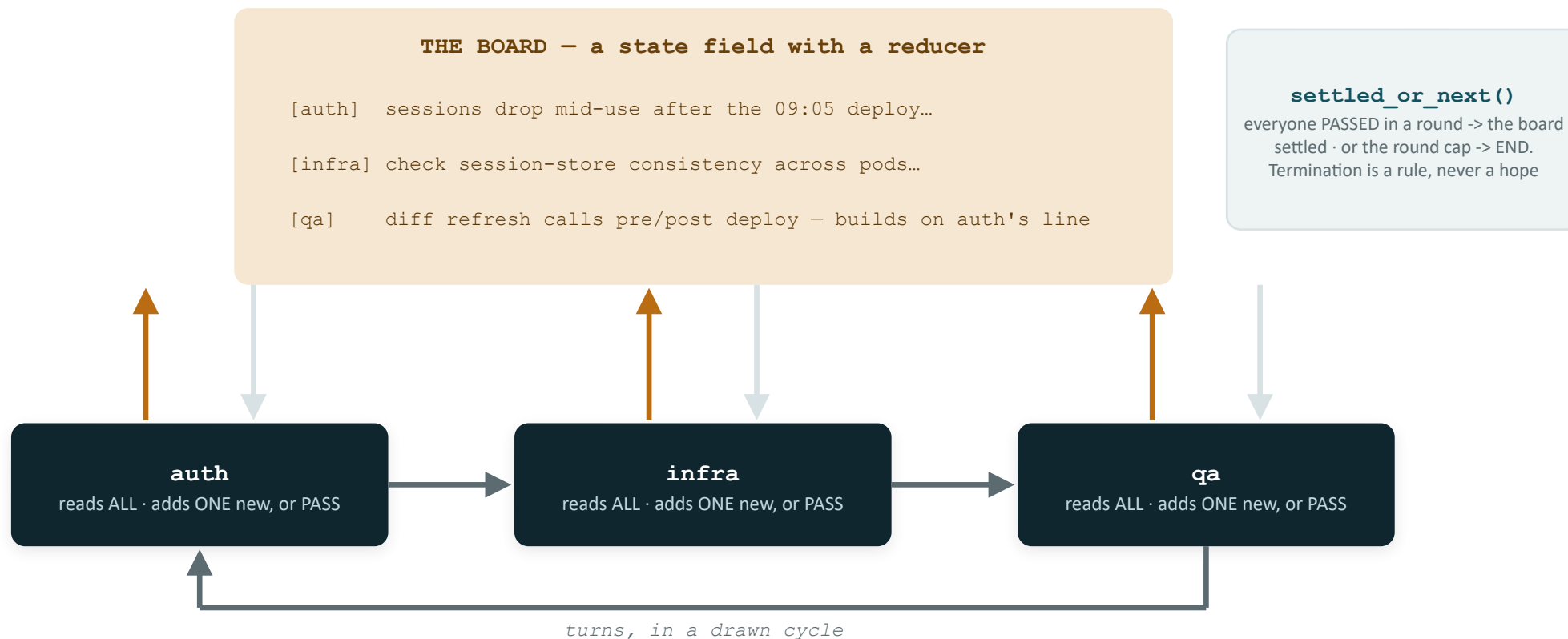
For growing ecosystems: next quarter's listener does not exist yet — and needs no rewiring.

DEMO `python p9_message_bus.py · ops events: alerts and deploys, different parties care`

Shared state — the board IS the graph state

Decoupled. LangGraph's core idea — state every node reads and writes — used as the whole pattern.

*no commander —
the BOARD is the
coordination*



Each finding is written against the board its author just READ — the two written rules kill the two failure modes: PASS beats padding, the cap beats loops.

Shared context with no natural owner — coordination through state alone. Eleven for eleven.

DEMO `python pl0_shared_state.py` · an incident hypothesis board, no incident commander

How to choose

Four questions — and a default.

Is the flow known in advance?

fixed flow: sequential · concurrent · generator-verifier

Must someone decide at runtime?

a deciding agent: handoff · group chat · orchestrator · magentic

Independent long work, or a growing ecosystem?

decoupled: teams · bus · shared state

Not sure?

orchestrator-subagent — the default, until it visibly struggles

"Start with the simplest pattern that could work, watch where it struggles, and evolve." Production systems combine patterns.

Before the fleet runs: the inventory

Every agent: an owner, and an identity of its own.

fleet/inventory.yaml — LINTED, NOT DECORATIVE

```
- agent: story-judge
  owner: abhimanyu
  identity: github-pat, fine-grained,
    Issues RW, ONE repo only
  cost_cap_usd_per_day: 2.00

- agent: triage
  owner: ... identity: ... cap: ...

6 agents · 3 mandatory fields each
```

IDENTITY — THE QUIET HEADLINE

Credentials that belong to THE AGENT,
not to a person.

Revoke the row: the agent stops.

No human loses any access.

The linter is the point:

"an agent without an owner and its own identity
does not go past this line." A build check, not a policy hope.

An inventory nobody checks rots into a wiki page. This one fails the build.

What to record on every run

The rule: a run that leaves no record did not happen.

ONE LINE PER RUN — fleet/records.jsonl

```
{ "run_id": "8f3a...", "agent": ...,  
  "prompt_version": "v2",  
  "model": ..., "tokens": ...,  
  "cost_usd": 0.0001, "seconds": ...,  
  "outcome": "FAIL"  <- a validator,  
  not an opinion }
```

"Quietly wrong" means: every output reads fine.

The drift is visible in exactly one place — the validator column of the record. Nowhere on the screen.

THE DRIFT, AS CAPTURED — LIVE RUNS

```
v1  'at most 20 words'  
    16 words pass · 17 pass · 18 pass
```

```
v2  someone 'improved' the brief  
    43 words FAIL · 42 FAIL · 44 FAIL  
    — and every output read fine
```

```
v3  the constraint restored  
    16 pass · 19 pass · 17 pass
```

v2->v3 is the ONE PROVED CHANGE:
a pass rate, not an argument

An agent run that leaves no record did not happen.

The fleet view — every agent, one screen

Read straight from the week's own records. Nothing invented.

python fleet/fleetview.py — AS CAPTURED

agent	day	runs	in_tok	out_tok	cost	\$/run	
test-designer	3	3	11022	4681	0.0744	0.0248	
triage	3	4	30380	3404	0.1100	0.0275	
knowledge-ask	3	1	2944	487	0.0098	0.0098	
note-summarizer	4	9	537	315	0.0002	0.0000	
p0 .. p10	4	11	15334	6520	0.0034	0.0003	<- all eleven patterns
TOTAL						0.1977	

COST OUTLIER triage at \$0.0275/run — orders of magnitude above the nano agents

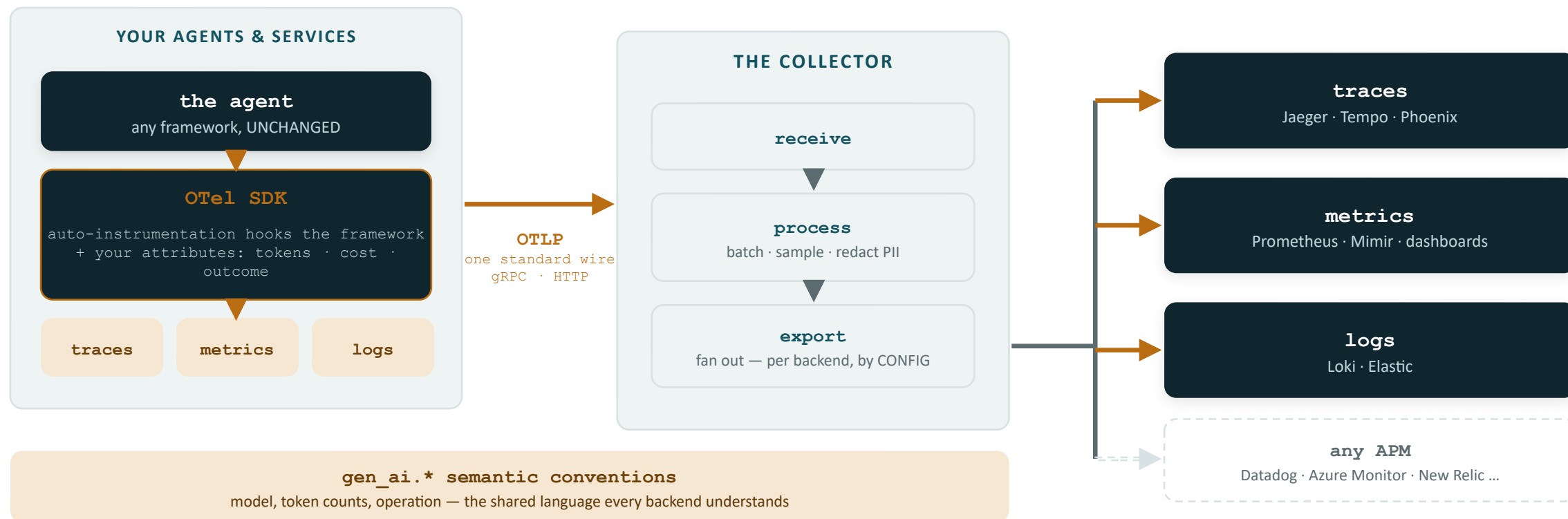
CHANGE TRACKING note-summarizer: v1 100% ##### v2 0% v3 100% #####

No dashboard product, no anomaly model — a division and a max over records you already had.

Cost, the outlier, the drift, and one proved change — one screen, because every run left a record.

OpenTelemetry — how the wiring works

Instrument once, one standard wire, process in a collector, choose backends by config.



Swapping or adding a backend touches only the exporter CONFIG — never the agents. (Our demo exports straight from the SDK — fine on one laptop; production adds the collector.)

Lock-in ends at the SDK: everything after the standard wire is configuration.

The fleet on the industry pipe — OpenTelemetry

Same evidence, standard plumbing. No demo file was touched.

THREE COMMANDS — ZERO DEMO EDITS

```
python otel/otel_run.py \
    patterns/p7_magentic.py
-> live trace in Phoenix :6006
python otel/backfill_ledger.py
-> the week's ledgers, replayed
python otel/otel_run.py ... --both
-> SAME run: Phoenix AND Grafana
```

Instrumentation is orthogonal to code.

One wrapper registers the OTEL pipe and hooks the LangChain callback layer — the demo never learns it is being watched.

AS CAPTURED

```
p7_magentic ONE trace, 29 spans:
manager x3 · read_logs · read_code
propose_fix · route_manager x3
fix_or_back <- the written rule,
                visible as a span
6 LLM calls · 3,181 tokens =
the ledger's 2,141+1,040 EXACTLY
```

```
fleet-week 30 recorded runs ->
30 spans: real timestamps, cost,
outcome — nothing re-run
```

```
--both one extra exporter;
vendor = config, not a rewrite
```

The ledger stays the source of truth. OTEL is the standard pipe from it to ANY backend.

Three things to carry

1

Start simple. Evolve on evidence.

Eleven patterns are one construction kit — `add_node`, `add_edge`, conditional edges, `Send`, `Command`, `state`. Orchestrator-subagent is the default; move on only when it visibly struggles.

2

Every agent gets an owner and an identity of its own.

A person who answers for it, credentials that are revocable without touching any human's access, and a cost cap. Enforced by a linter, not hoped for in a wiki.

3

A run that leaves no record did not happen.

Records made the quietly-wrong agent visible and the fix provable — and they made the fleet view free. One JSON line per run buys all of it.

Questions — and thank you for the four days. The decks, fixtures and run sheets are yours.